

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph and Tree

Oleh:

Erina Aprilia

NIM. 2510817120020

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph and Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Erina Aprilia
NIM : 2510817120020

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	i
DAFTAR ISI	ii
DAFTAR GAMBAR.....	iii
DAFTAR TABEL	iv
Soal 1: Konversi Adjacency List ke Adjacency Matrix	5
A. Source Code.....	7
B. Output Program.....	8
C. Pembahasan	8
Soal 2: Konversi Adjacency Matrix ke Adjacency List	11
A. Source Code.....	12
B. Output Program.....	14
C. Pembahasan	14
Soal 3: BFS: Jarak Terpendek Keluar Labirin.....	16
A. Source Code.....	18
B. Output Program.....	19
C. Pembahasan	19
Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin.....	21
A. Source Code.....	23
B. Output Program.....	24
C. Pembahasan	24
Soal 5: Tree Traversal.....	26
A. Source Code.....	27
B. Output Program.....	30
C. Pembahasan	30
TAUTAN GIT	32

DAFTAR GAMBAR

Gambar 1 output soal1.cpp	8
Gambar 2 output soal2.cpp	14
Gambar 3 output soal3.cpp	19
Gambar 4 output soal4.cpp	24
Gambar 5 output soal5.cpp	30

DAFTAR TABEL

Tabel 1. soal1.cpp	7
Tabel 2. soal2.cpp	12
Tabel 3. soal3.cpp	18
Tabel 4. soal4.cpp	23
Tabel 5. searching_algorithm.cpp	27

Soal 1: Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U,V,W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U,V,W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi **adjacency matrix**. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
M
U1 V1 W1
U2
V2
W
2
....
UM VM WM
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

```
V1 V2 ... VN
V1 a11 a12 ... a1N
V2 a21 a22 ... a1N
...
VN aN1 aN2 ... aNN
```

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki **self-loop**.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 0 0 0 0 0

Penjelasan Contoh

Edge A B 4 menghasilkan nilai 4 pada baris A kolom B. Karena graph berarah, nilai pada baris B kolom A tetap 0 selama tidak ada edge dari B menuju A.

Edge D C 3 menghasilkan nilai 3 pada baris D kolom C. Vertex E tidak memiliki edge keluar, sehingga seluruh nilai pada baris E adalah 0.

A. Source Code

Tabel 1 soal1.cpp

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  int main() {
6      int N;
7      cin >> N;
8
9      string labels[10];
10     for (int i = 0; i < N; i++) {
11         cin >> labels[i];
12     }
13
14     int matrix[10][10] = {0};
15
16     int M;
17     cin >> M;
18
19     for (int i = 0; i < M; i++) {
20         string U, V;
21         int W;
22         cin >> U >> V >> W;
23
24         int idxU = -1, idxV = -1;
25         for (int j = 0; j < N; j++) {
26             if (labels[j] == U) idxU = j;
27             if (labels[j] == V) idxV = j;
28         }
29
30         matrix[idxU][idxV] = W;
31     }
32
33     int colWidth = 1;
34     for (int i = 0; i < N; i++) {
35         if ((int)labels[i].size() > colWidth)
36             colWidth = labels[i].size();
37     }
38     for (int i = 0; i < N; i++) {
39         for (int j = 0; j < N; j++) {
40             string s = to_string(matrix[i][j]);
41             if ((int)s.size() > colWidth)
42                 colWidth = s.size();
43         }
```



```

44     }
45
46     cout << "Adjacency Matrix:" << endl;
47     cout << string(colWidth, ' ');
48     for (int j = 0; j < N; j++) {
49         cout << " ";
50         cout << string(colWidth - labels[j].size(), ' ')
<< labels[j];
51     }
52     cout << endl;
53
54     for (int i = 0; i < N; i++) {
55         cout << string(colWidth - labels[i].size(), ' ')
<< labels[i];
56         for (int j = 0; j < N; j++) {
57             string val = to_string(matrix[i][j]);
58             cout << " " << string(colWidth - val.size(),
' ') << val;
59         }
60         cout << endl;
61     }
62
63     return 0;
64 }

```

B. Output Program

```

PS C:\Users\HYPE AMD\module-6-graph-and-tree-alyrein847-jpg> cd "c:\Users\HYPE AMD\module-6-graph-and-tree-alyrein847-jpg\" ; if (
$?) { g++ soal1.cpp -o soal1 } ; if ($?) { .\soal1 }
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
PS C:\Users\HYPE AMD\module-6-graph-and-tree-alyrein847-jpg>

```

Gambar 1 output soal1.cpp

C. Pembahasan

Soal ini meminta hasil akhir berupa matriks berukuran $N \times N$, di mana $matrix[i][j]$ berisi bobot edge dari vertex ke- i menuju vertex ke- j , dan bernilai 0 jika edge tersebut tidak

ada. Namun, input yang diberikan berupa label vertex dalam bentuk huruf, bukan dalam bentuk index numerik. Karena terdapat perbedaan bentuk antara data masukan (label) dan bentuk yang diminta sebagai keluaran (index pada matriks), maka langkah penyelesaian yang diambil adalah membentuk pemetaan dari label ke index terlebih dahulu berdasarkan urutan kemunculannya pada input, sebelum data edge tersebut dapat dimasukkan ke dalam matriks.

Berdasarkan hal tersebut, langkah pertama pada program adalah membaca seluruh label vertex dan menyimpannya ke dalam array `labels[]` sesuai urutan input. Posisi suatu label pada array inilah yang kemudian menjadi index vertex tersebut di matriks. Setelah itu, program membaca M edge satu per satu. Untuk setiap edge (U, V, W) , program perlu menentukan posisi index dari label U dan label V dengan cara memeriksa array `labels[]` dari awal hingga ditemukan label yang sesuai. Pencarian ini pada dasarnya adalah linear search, tapi data yang dibandingkan berupa string label, bukan angka. Setelah index U (`idxU`) dan index V (`idxV`) diperoleh, nilai bobot W langsung dimasukkan ke `matrix[idxU][idxV]`. Karena graph yang dimaksud bersifat berarah, hanya satu sel tersebut yang diisi, tanpa mengisi sel sebaliknya (`matrix[idxV][idxU]`), karena edge dari A ke B tidak berarti terdapat edge dari B ke A .

Bagian program setelah pengisian matriks selanjutnya berkaitan dengan penyesuaian format keluaran. Karena bobot W dapat memiliki hingga 4 digit sedangkan label hanya terdiri dari 1 karakter, jadi diperlukan penyamaan lebar kolom agar tampilan matriks tetap rapi. Untuk itu, program menghitung panjang string terbesar di antara seluruh label dan seluruh nilai pada matriks, kemudian menyimpannya sebagai `colWidth`. Nilai ini kemudian digunakan untuk menambahkan spasi pada setiap label dan angka saat dicetak, sehingga seluruh kolom tercetak rata kanan dengan lebar yang konsisten.

Struktur `matrix[10][10]` yang digunakan didasarkan pada batasan soal, yaitu N tidak lebih dari 10, sehingga ukuran maksimum matriks sudah dapat ditentukan sejak awal tanpa perlu berubah ukuran. Selain itu, deklarasi `int matrix[10][10] = {0}` digunakan untuk membuat seluruh sel matriks otomatis terisi nilai 0 sejak awal, agar tidak diperlukan proses inisialisasi nilai nol secara manual.

Dari sisi kompleksitas, terdapat tiga proses utama yang menyumbang jumlah operasi terbanyak. Yaitu, pembacaan N label yang membutuhkan $O(N)$, pemrosesan M edge yang masing-masing memerlukan pencarian index U dan index V melalui linear search (dengan kompleksitas hingga $O(N)$ per pencarian), serta pencetakan matriks yang harus mengakses seluruh $N \times N$ sel. Proses pencarian index inilah yang memiliki jumlah operasi paling besar, karena dilakukan dua kali untuk setiap edge, sehingga totalnya mencapai $O(M \times N)$. Dengan M dapat bernilai hingga $N \times (N-1)$, maka kompleksitas waktu keseluruhan pada kondisi terburuk dapat mendekati $O(N^3)$. Namun, karena batasan N pada soal ini maksimal hanya 10, jumlah operasi yang terjadi tetap sangat kecil meskipun jumlah edge sudah mencapai batas maksimum (90 edge), sehingga

pendekatan linear search untuk pencarian index masih sesuai digunakan tanpa memerlukan struktur tambahan yang lebih kompleks.

Dari sisi kompleksitas ruang, matriks yang digunakan selalu berukuran tetap sebesar $N \times N$, tidak bergantung pada jumlah edge yang sebenarnya ada. Pada contoh soal misalnya, dengan 5 vertex dan 6 edge, matriks yang dialokasikan tetap berukuran 25 sel, bukan hanya sejumlah edge yang tercatat. Hasil tersebut sesuai dengan karakteristik adjacency matrix, karena kebutuhan ruang penyimpanannya bergantung pada jumlah vertex yang digunakan. Oleh sebab itu, meskipun jumlah edge bertambah atau berkurang, ukuran matriks yang dialokasikan tetap ditentukan oleh banyaknya vertex pada graf.

Soal 2: Konversi Adjacency Matrix ke Adjacency List

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang direpresentasikan dalam bentuk **adjacency matrix** berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi **adjacency list**. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
A11 A12 ... A1N
A21 A22
... A2N ...
AN1 AN2 ... ANN
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

```
V1 -> (X1,w1), (X2,w2), ...
```

V2 -> (X1,w1), (X2,w2), ...

...

... VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
Semua label vertex dijamin unik.
Nilai diagonal utama matrix dijamin 0. Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B,4), (C,2) B -> (D,7) C -> (B,1), (E,6) D -> (C,3) -> -

Penjelasan Contoh

Pada baris vertex A, nilai positif muncul pada kolom B dan C. Karena itu, adjacency list untuk A adalah (B,4), (C,2).

Pada baris vertex E, semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah E -> -.

A. Source Code

Tabel 2 soal2.cpp

1	#include <iostream>
2	#include <string>
3	#include <bits/stdc++.h>
4	using namespace std;

```

5
6 int main() {
7     int n;
8     cin >> n;
9
10    vector<string> label(n);
11    for (int i = 0; i < n; i++) {
12        cin >> label[i];
13    }
14
15    vector<vector<int>> mat(n, vector<int>(n));
16    for (int i = 0; i < n; i++) {
17        for (int j = 0; j < n; j++) {
18            cin >> mat[i][j];
19        }
20    }
21
22    cout << "Adjacency List:" << "\n";
23
24    for (int i = 0; i < n; i++) {
25        cout << label[i] << " -> ";
26
27        bool adaEdge = false;
28        string hasil = "";
29
30        for (int j = 0; j < n; j++) {
31            if (mat[i][j] > 0) {
32                if (adaEdge) {
33                    hasil += ", ";
34                }
35                hasil += "(" + label[j] + ", " +
to_string(mat[i][j]) + ")";
36                adaEdge = true;
37            }
38        }
39
40        if (adaEdge) {
41            cout << hasil << "\n";
42        } else {
43            cout << "-" << "\n";
44        }
45    }
46
47    return 0;
48 }

```

B. Output Program

```
PS C:\Users\HYPE AMD\module-6-graph-and-tree-alyrein847-jpg> cd "c:\Users\HYPE AMD\module-6-graph-and-tree-alyrein847-jpg\" ; if ($?) { g++ soal2.cpp -o soal2 } ; if ($?) { .\soal2 }
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
PS C:\Users\HYPE AMD\module-6-graph-and-tree-alyrein847-jpg>
```

Gambar 2 output soal2.cpp

C. Pembahasan

Soal ini meminta keluaran berupa adjacency list, yaitu untuk setiap vertex perlu dicetak daftar vertex tujuan yang terhubung langsung beserta bobotnya. Berbeda dengan Soal 1, di sini data masukan sudah berbentuk matriks yang tersusun berdasarkan index, sehingga tidak diperlukan proses pemetaan label ke index seperti pada Soal 1. Karena bentuk data masukan dan tujuan keluarannya sudah sejalan dari sisi index, penyelesaian dapat langsung dilakukan dengan menyusuri matriks tersebut baris per baris untuk membentuk daftar tujuan dari setiap vertex.

Program pertama-tama membaca N label vertex ke dalam label[], kemudian membaca matriks $N \times N$ ke dalam mat[][]. Karena urutan baris dan kolom matriks pada input sudah mengikuti urutan label yang sama, index i dan j pada matriks otomatis merepresentasikan vertex ke-i dan ke-j tanpa perlu pencarian tambahan. Bagian utama program berada pada loop luar yang berjalan untuk setiap vertex i. Untuk setiap vertex tersebut, program memeriksa seluruh kolom j dari 0 hingga $N-1$ pada baris i. Jika $\text{mat}[i][j] > 0$, artinya terdapat edge dari vertex i ke vertex j dengan bobot sebesar $\text{mat}[i][j]$, dan pasangan tersebut kemudian disusun menjadi string berformat (label,bobot) yang ditambahkan ke variabel hasil. Variabel boolean adaEdge digunakan untuk menandai apakah vertex i sudah memiliki minimal satu edge keluar yang ditemukan, dan dipakai untuk menentukan apakah perlu ditambahkan tanda koma sebelum pasangan berikutnya. Setelah seluruh kolom diperiksa, jika adaEdge masih bernilai false, artinya vertex tersebut tidak memiliki edge keluar sama sekali, sehingga program mencetak tanda "-" sebagai gantinya.

Dari sisi kompleksitas waktu, proses pembacaan label membutuhkan $O(N)$, sedangkan pembacaan matriks membutuhkan $O(N^2)$ karena harus mengisi seluruh sel. Pada bagian pembentukan adjacency list, program melakukan loop sebanyak N kali untuk setiap vertex, dan di dalamnya terdapat loop lagi sebanyak N kali untuk memeriksa seluruh kolom pada baris tersebut. Karena pemeriksaan ini dilakukan untuk seluruh sel matriks

tanpa terkecuali, kompleksitas waktu pada bagian ini adalah $O(N^2)$, sehingga total kompleksitas waktu program secara keseluruhan adalah $O(N^2)$.

Jika dibandingkan dengan Soal 1, kompleksitas pada Soal 2 ini sebenarnya lebih sederhana. Pada Soal 1, setiap edge yang dibaca masih harus dicari dahulu index-nya melalui linear search terhadap label, sehingga muncul tambahan faktor pencarian yang membuat kompleksitasnya bisa mendekati $O(N^3)$ pada kasus terburuk. Sementara pada Soal 2, karena data masukan sudah berbentuk matriks yang tersusun berdasarkan index, tidak ada lagi proses pencarian semacam itu, sehingga kompleksitasnya turun menjadi $O(N^2)$ saja. Hal ini menunjukkan bahwa bentuk representasi data masukan turut menentukan seberapa besar usaha tambahan yang diperlukan untuk mengubahnya ke representasi lain.

Dari sisi kompleksitas ruang, program ini menggunakan `vector<string>` berukuran N untuk label dan `vector<vector<int>>` berukuran $N \times N$ untuk matriks, sehingga total ruang yang dibutuhkan adalah $O(N^2)$, didominasi oleh penyimpanan matriks. Selain itu, terdapat variabel hasil berupa string yang dibentuk ulang setiap kali program berpindah ke vertex baru, namun ukurannya tidak memengaruhi kompleksitas ruang secara keseluruhan karena tetap terikat pada jumlah edge keluar dari satu vertex, yang paling banyak sejumlah $N-1$.

Penggunaan vector pada Soal 2 (dibandingkan array statis seperti pada Soal 1) sendiri merupakan pilihan implementasi, bukan keharusan dari batasan soal, karena N pada soal ini juga dibatasi maksimal 10. Namun pendekatan ini tetap valid karena ukuran vector ditentukan langsung dari nilai N yang dibaca di awal, sehingga tidak ada risiko ukuran data yang melebihi alokasi.

Soal 3: BFS: Jarak Terpendek Keluar Labirin

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma **Breadth-First Search** (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22
... G2C ...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
 $1 \leq C \leq 100$
 G_{ij} hanya bernilai 0 atau 1.
 $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9	16

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (7,9). Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel (2,1).

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari (0,0) ke (7,9) adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Tabel 3 soal3.cpp

```
1  #include <iostream>
2  #include <string>
3  #include <bits/stdc++.h>
4  using namespace std;
5
6  int main() {
7      int r, c;
8      cin >> r >> c;
9
10     vector<vector<int>> grid(r, vector<int>(c));
11     for (int i = 0; i < r; i++) {
12         for (int j = 0; j < c; j++) {
13             cin >> grid[i][j];
14         }
15     }
16
17     int sr, sc, fr, fc;
18     cin >> sr >> sc;
19     cin >> fr >> fc;
20
21     vector<vector<int>> dist(r, vector<int>(c, -1));
22
23     queue<pair<int, int>> q;
24     dist[sr][sc] = 0;
25     q.push({sr, sc});
26
27     int dr[4] = {-1, 1, 0, 0};
28     int dc[4] = {0, 0, -1, 1};
29
30     while (!q.empty()) {
31         pair<int, int> cur = q.front();
32         q.pop();
33         int cr = cur.first;
34         int cc = cur.second;
35
36         if (cr == fr && cc == fc) {
37             break;
38         }
```

```

39
40         for (int d = 0; d < 4; d++) {
41             int nr = cr + dr[d];
42             int nc = cc + dc[d];
43
44             bool valid = (nr >= 0 && nr < r && nc >= 0
&& nc < c);
45             if (valid && grid[nr][nc] == 0 &&
dist[nr][nc] == -1) {
46                 dist[nr][nc] = dist[cr][cc] + 1;
47                 q.push({nr, nc});
48             }
49         }
50     }
51
52     cout << dist[fr][fc] << "\n";
53     return 0;
54 }

```

B. Output Program

```

PS C:\Users\HYPE AMD\module-6-graph-and-tree-alyrein847-jpg> cd "c:\Users\HYPE AMD\module-6-graph-and-tree-alyrein847-jpg\" ; if (
$?) { g++ soal3.cpp -o soal3 } ; if ($?) { .\soal3 }
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
PS C:\Users\HYPE AMD\module-6-graph-and-tree-alyrein847-jpg>

```

Gambar 3 output soal3.cpp

C. Pembahasan

Soal ini meminta jarak minimum dari suatu posisi awal ke posisi tujuan di dalam sebuah labirin berbentuk grid $R \times C$. Karena yang dicari adalah jarak terpendek dan setiap langkah memiliki biaya yang sama, BFS menjadi pilihan yang tepat karena cara kerjanya yang menyebar lapis demi lapis dari titik awal memastikan setiap sel yang pertama kali dijangkau selalu melalui jalur terpendek.

Program menggunakan antrian untuk menyimpan sel-sel yang akan diproses, serta matriks dist berukuran $R \times C$ untuk mencatat jarak dari posisi awal ke setiap sel. Seluruh sel pada dist diisi -1 sebagai tanda belum pernah dikunjungi, lalu sel awal diisi 0 dan dimasukkan ke antrian sebagai titik mulai.

Program kemudian memproses antrian satu per satu. Setiap sel yang diambil dari antrian akan diperiksa keempat tetangganya menggunakan array arah $dr[]$ dan $dc[]$. Jika tetangga tersebut masih dalam batas grid, bukan dinding, dan belum pernah dikunjungi, maka jaraknya diisi dengan jarak sel saat ini ditambah 1, lalu dimasukkan ke antrian. Proses ini terus berjalan sampai antrian habis atau sel tujuan sudah ditemukan. Hasil akhirnya langsung diambil dari $dist[fr][fc]$, dan karena nilai -1 tidak pernah diubah untuk sel yang tidak terjangkau, output otomatis menjadi -1 jika memang tidak ada jalur yang bisa dilalui.

Dari sisi kompleksitas waktu, setiap sel hanya masuk ke antrian paling banyak satu kali karena langsung ditandai saat pertama dikunjungi, sehingga kompleksitas waktunya adalah $O(R \times C)$. Pemeriksaan 4 tetangga per sel hanya menambah konstanta yang tidak mempengaruhi orde kompleksitasnya.

Untuk kompleksitas ruang, program menyimpan dua struktur berukuran $R \times C$ yaitu grid dan dist, ditambah antrian yang dalam kondisi terburuk bisa menampung seluruh sel yang terjangkau, sehingga kompleksitas ruangnya juga $O(R \times C)$.

Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma **Depth-First Search** (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses **backtracking** agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22
... G2C ...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $\leq R \leq 7$

$$1 \leq C \leq 7$$

G_{ij} hanya bernilai 0 atau 1.

$$0 \leq S_R < R$$

- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (4,5). Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel (0,5).

Dengan DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

A. Source Code

Tabel 4. soal4.cpp

```
1  #include <iostream>
2  #include <string>
3  #include <bits/stdc++.h>
4  using namespace std;
5
6  int R, C;
7  vector<vector<int>> grid;
8  vector<vector<bool>> visited;
9  int FR, FC;
10 long long jumlahJalur = 0;
11
12 int dr[4] = {-1, 1, 0, 0};
13 int dc[4] = {0, 0, -1, 1};
14
15 void dfs(int r, int c) {
16     if (r == FR && c == FC) {
17         jumlahJalur++;
18         return;
19     }
20
21     visited[r][c] = true;
22
23     for (int d = 0; d < 4; d++) {
24         int nr = r + dr[d];
25         int nc = c + dc[d];
26
27         bool valid = (nr >= 0 && nr < R && nc >= 0 && nc
28 < C);
29         if (valid && grid[nr][nc] == 0 &&
30 !visited[nr][nc]) {
31             dfs(nr, nc);
32         }
33     }
34     visited[r][c] = false;
35 }
36
37 int main() {
38     cin >> R >> C;
39
40     grid.assign(R, vector<int>(C));
41     for (int i = 0; i < R; i++) {
42         for (int j = 0; j < C; j++) {
```



```

42         cin >> grid[i][j];
43     }
44 }
45
46     int sr, sc;
47     cin >> sr >> sc;
48     cin >> FR >> FC;
49
50     visited.assign(R, vector<bool>(C, false));
51     dfs(sr, sc);
52
53     cout << jumlahJalur << "\n";
54     return 0;
55 }

```

B. Output Program

```

PS C:\Users\HYPE AMD\module-6-graph-and-tree-alyrein847-jpg> cd "c:\Users\HYPE AMD\module-6-graph-and-tree-alyrein847-jpg\" ; if (
$?) { g++ soal4.cpp -o soal4 } ; if ($?) { .\soal4 }
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
PS C:\Users\HYPE AMD\module-6-graph-and-tree-alyrein847-jpg>

```

Gambar 4 output soal4.cpp

C. Pembahasan

Soal ini meminta jumlah seluruh jalur sederhana dari posisi awal ke posisi tujuan pada labirin yang sama seperti soal sebelumnya. Berbeda dengan soal 3 yang hanya mencari satu jalur terpendek, di sini semua kemungkinan jalur harus ditemukan dan dihitung. Karena itu, DFS dengan backtracking menjadi pendekatan yang sesuai program perlu menjelajahi satu jalur sampai mentok atau sampai tujuan, lalu kembali dan mencoba arah lain yang belum pernah dicoba.

Program menggunakan fungsi rekursif `dfs(r, c)` yang dipanggil dari posisi awal. Setiap kali fungsi dipanggil, program pertama kali memeriksa apakah posisi saat ini sudah sampai di tujuan. Jika iya, `jumlahJalur` ditambah satu dan fungsi langsung kembali tanpa meneruskan pencarian lebih jauh. Jika belum, sel saat ini ditandai sebagai sudah dikunjungi di `visited[][]`, lalu program mencoba bergerak ke keempat arah. Untuk setiap arah yang valid masih dalam batas grid, bukan dinding, dan belum dikunjungi fungsi `dfs` dipanggil lagi secara rekursif. Setelah seluruh arah selesai dicoba, tanda kunjungan pada sel saat ini dihapus kembali. Inilah bagian backtracking-nya: dengan

mengembalikan status sel ke belum dikunjungi setelah selesai diproses, program memungkinkan sel yang sama untuk dilewati lagi pada jalur lain yang berbeda.

Dari sisi kompleksitas waktu, ini yang menjadi tantangan terbesar pada soal ini. Dalam kondisi terburuk, yaitu labirin yang seluruh selnya terbuka tanpa dinding, jumlah jalur sederhana yang mungkin ada bisa tumbuh sangat cepat seiring bertambahnya ukuran grid. Karena setiap jalur bisa melewati kombinasi sel yang berbeda-beda, kompleksitas waktu DFS dengan backtracking pada kasus ini secara teoritis bisa mencapai $O(R \times C)!$ dalam skenario paling ekstrem. Itulah mengapa batasan soal membatasi ukuran grid maksimal hanya 7×7 dan menjamin jumlah jalur valid tidak lebih dari 100.000, agar waktu eksekusi tetap terkendali.

Untuk kompleksitas ruang, program menyimpan grid dan visited masing-masing berukuran $R \times C$. Selain itu, karena DFS berjalan secara rekursif, terdapat call stack yang kedalamannya bisa mencapai $R \times C$ pada kondisi terburuk, yaitu ketika jalur yang sedang dijelajahi melewati seluruh sel grid. Sehingga kompleksitas ruang keseluruhannya adalah $O(R \times C)$, dengan catatan bahwa kedalaman rekursi juga ikut berkontribusi pada penggunaan memori di stack.

Soal 5: Tree Traversal

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah **RedBlack Tree** secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.

ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N
A1 A2 ... AN
Q
T1
T
2
.
.
.
TQ

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar_angka

[Inorder] : daftar_angka

[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$
- $-10000000000 \leq A_i \leq 10000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDER	[Preorder] : 50 30 20 40 70 60 80
INORDER	[Inorder] : 20 30 40 50 60 70 80
POSTORDER	[Postorder] : 20 40 30 60 80 70 50
ALL	

Penjelasan Contoh

Bilangan dimasukkan ke RBTree secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan.

Hasil INORDER pada RBTree selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTree yang dibentuk sudah konsisten dengan aturan RBTree.

A. Source Code

Tabel 5 Soal5.cpp

1	#include <iostream>
2	#include <string>
3	#include "RedBlackTree.h"

```

4  #include <bits/stdc++.h>
5  using namespace std;
6
7  using Node = RedBlackTree::Node;
8
9  void preorder(const Node* node, const Node* nil,
10 vector<int>& out) {
11     if (node == nil) return;
12     out.push_back(node->key);
13     preorder(node->left, nil, out);
14     preorder(node->right, nil, out);
15 }
16
17 void inorder(const Node* node, const Node* nil,
18 vector<int>& out) {
19     if (node == nil) return;
20     inorder(node->left, nil, out);
21     out.push_back(node->key);
22     inorder(node->right, nil, out);
23 }
24
25 void postorder(const Node* node, const Node* nil,
26 vector<int>& out) {
27     if (node == nil) return;
28     postorder(node->left, nil, out);
29     postorder(node->right, nil, out);
30     out.push_back(node->key);
31 }
32
33 void printList(const string& label, const vector<int>&
34 list) {
35     cout << label << " : ";
36     for (size_t i = 0; i < list.size(); i++) {
37         if (i > 0) cout << " ";
38         cout << list[i];
39     }
40     cout << "\n";
41 }
42
43 int main() {
44     int n;
45     cin >> n;
46
47     RedBlackTree tree;
48     for (int i = 0; i < n; i++) {

```

```

44         int value;
45         cin >> value;
46
47         if (!tree.contains(value)) {
48             tree.insert(value);
49         }
50     }
51
52     int q;
53     cin >> q;
54
55     vector<string> queries(q);
56     for (int i = 0; i < q; i++) {
57         cin >> queries[i];
58     }
59
60     if (tree.empty()) {
61         cout << "Tree kosong. Tidak ada yang bisa
62 ditampilkan." << "\n";
63         return 0;
64     }
65
66     const Node* root = tree.root();
67     const Node* nilNode = tree.nil();
68
69     for (const string& type : queries) {
70         if (type == "PREORDER" || type == "ALL") {
71             vector<int> pre;
72             preorder(root, nilNode, pre);
73             printList("[Preorder]", pre);
74         }
75         if (type == "INORDER" || type == "ALL") {
76             vector<int> in;
77             inorder(root, nilNode, in);
78             printList("[Inorder]", in);
79         }
80         if (type == "POSTORDER" || type == "ALL") {
81             vector<int> post;
82             postorder(root, nilNode, post);
83             printList("[Postorder]", post);
84         }
85     }
86
87     return 0;
88 }

```

B. Output Program

```
PS C:\Users\HYPE_AMD\module-6-graph-and-tree-alyrein847-jpg> g++ -o soal5 soal5.cpp RedBlackTree.cpp
>> ./soal5
7
50 30 70 20 40 60 80
4
PREORDER
INORDER
POSTORDER
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
PS C:\Users\HYPE_AMD\module-6-graph-and-tree-alyrein847-jpg>
```

Gambar 5 output soal5.cpp

C. Pembahasan

Output yang diharapkan dari soal ini adalah hasil traversal dari sebuah Red-Black Tree yang dibentuk dari N bilangan bulat yang diberikan sebagai input. Jika ada bilangan yang sama, bilangan tersebut diabaikan dan tidak dimasukkan ke tree. Setelah tree terbentuk, program menerima Q query yang masing-masing meminta hasil traversal PREORDER, INORDER, POSTORDER, atau ALL yang mencetak ketiganya sekaligus.

Karena struktur Red-Black Tree sudah disediakan, Langkah selanjutnya hanya mengisi tree dari input, lalu menelusuri hasilnya sesuai query. Pengisian tree dilakukan dengan memanggil `tree.insert(value)` untuk setiap bilangan, namun sebelumnya dicek dulu dengan `tree.contains(value)` agar tidak ada duplikat yang masuk.

Untuk traversalnya, program menggunakan tiga fungsi rekursif terpisah yaitu preorder, inorder, dan postorder. Ketiganya bekerja dengan cara yang serupa menelusuri node secara rekursif sambil mengumpulkan nilai ke dalam vector out, namun berbeda pada urutan kapan nilai node saat ini dimasukkan. Pada preorder, nilai node dimasukkan lebih dulu sebelum menelusuri subtree kiri dan kanan. Pada inorder, nilai node baru dimasukkan setelah subtree kiri selesai ditelusuri. Sedangkan pada postorder, nilai node baru dimasukkan paling akhir setelah kedua subtree selesai. Ketiga fungsi ini berhenti ketika node yang diterima adalah `nilNode`, yaitu sentinel node milik Red-Black Tree yang menandai ujung dari setiap cabang.

Satu hal yang menarik dari Red-Black Tree adalah bahwa hasil INORDER-nya selalu menghasilkan urutan nilai dari kecil ke besar. Ini bukan kebetulan, melainkan konsekuensi langsung dari sifat BST yang mendasari Red-Black Tree, di mana setiap node di subtree kiri selalu lebih kecil dari node saat ini, dan setiap node di subtree kanan selalu lebih besar. Traversal inorder yang mengunjungi kiri → root → kanan otomatis menghasilkan urutan yang tersorting.

Dari sisi kompleksitas waktu, proses insert N bilangan ke Red-Black Tree masing-masing membutuhkan $O(\log N)$ karena tree selalu menjaga keseimbangannya setelah setiap operasi insert. Sehingga total waktu pengisian tree adalah $O(N \log N)$. Untuk setiap query traversal, seluruh node dikunjungi tepat satu kali, sehingga kompleksitasnya adalah $O(N)$ per traversal. Dengan Q query, total waktu untuk menjawab seluruh query adalah $O(Q \times N)$, dan keseluruhan kompleksitas waktu program menjadi $O(N \log N + Q \times N)$.

Untuk kompleksitas ruang, tree menyimpan N node secara keseluruhan. Setiap pemanggilan fungsi traversal menggunakan call stack yang kedalamannya setara dengan tinggi tree, yaitu $O(\log N)$ karena Red-Black Tree selalu seimbang. Selain itu, vector out yang menampung hasil traversal berukuran $O(N)$. Sehingga kompleksitas ruang keseluruhannya adalah $O(N)$.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-alyrein847-jpg>